

RAPPORT D'AUDIT DE SÉCURITÉ

MAXIMUS ERP

Code, dépendances, GitHub, Render et production

21 septembre 2026

Audit non destructif — aucun secret ni token inclus dans ce document

Conclusion exécutive

La production est opérationnelle et aucun problème critique n'a été détecté. Les priorités sont la mise à jour de plusieurs dépendances, la protection de la branche principale GitHub, l'activation de Dependabot/CodeQL et l'alignement de la configuration Render.

1. Résumé exécutif

L'audit a couvert le code MAXIMUS Laravel/React, les dépendances npm et Composer, les scanners de sécurité, les règles d'authentification et d'isolation des entreprises, le dépôt GitHub, la configuration Render et les réponses de l'API de production. L'audit n'a effectué aucune écriture sur GitHub, Render, la base de données ou les secrets.

Vulnérabilités critiques	0
Vulnérabilités hautes	1
Vulnérabilités modérées	3
Vulnérabilités faibles	2
SAST	Aucun résultat
HoundDog / flux sensibles	Aucun résultat
Composer	Aucune vulnérabilité connue
Production Render	Opérationnelle
Alertes de secrets GitHub	Aucune alerte ouverte

2. Résultats prioritaires

2.1 — Haute priorité : js-yaml 4.3.1

La dépendance js-yaml 4.3.1 est utilisée par l'outillage orval dans lib/api-spec. L'audit de dépendances identifie l'avis GHSA-2883-xcg3-v3hh. Une version corrigée 4.3.2 est disponible. L'exposition directe en production semble limitée car cette dépendance est rattachée à l'outillage de développement, mais elle est présente dans le lockfile du dépôt public.

Action : mettre à jour js-yaml vers 4.3.2 ou une version compatible plus récente, puis relancer les tests, le build et l'audit.

2.2 — Priorité modérée : qs 6.15.3

qs 6.15.3 est utilisé dans la chaîne API via express et body-parser. Deux avis concernent cette version. La version corrigée indiquée par le scanner est 6.16.0. Cette dépendance est plus importante que les outils de build car elle se trouve dans le chemin de production de l'API.

Action : mettre à jour qs vers 6.16.0, vérifier les tests des paramètres HTTP et reconstruire l'image Render.

2.3 — Priorité modérée/faible : esbuild et nanoid

Le lockfile contient plusieurs versions d'esbuild, dont une ancienne 0.18.20. La résolution active des workspaces utilise principalement 0.27.3. Le scanner signale aussi esbuild 0.27.3 et nanoid 3.3.16, tous deux utilisés par l'outillage Vite/PostCSS.

Action : nettoyer les anciennes résolutions du lockfile et mettre à jour esbuild vers une version corrigée ainsi que nanoid vers 3.3.18, sans contourner le pare-feu de paquets.

3. GitHub

Le dépôt maoddoka-wq/Maximus est public. La branche main n'est pas protégée et aucun mécanisme de validation obligatoire avant fusion n'a été détecté. Dependabot et les mises à jour de sécurité Dependabot sont désactivés. GitHub Code Scanning ne dispose d'aucune analyse configurée.

- Point positif : Secret Scanning est activé.
- Point positif : Push Protection est activée.
- Point positif : aucune alerte de secret ouverte n'a été retournée.
- Le compte interrogé est le seul collaborateur détecté et dispose des droits administrateur.
- Le dernier commit GitHub correspond au commit servi par Render.

Actions recommandées : protéger main, exiger une pull request et des contrôles réussis, activer Dependabot et configurer CodeQL.

4. Render et production

Le service Render maximus-erp est public, déployé automatiquement depuis main, avec un build réussi. L'endpoint /api/healthz a répondu 200 et a confirmé l'état opérationnel de la base PostgreSQL.

APP_DEBUG	false
Session cookie	Secure, HttpOnly, SameSite=Lax ; session serveur hashée
En-têtes	CSP, HSTS, X-Frame-Options DENY, nosniff et Referrer-Policy présents
Santé production	HTTP 200 ; base de données opérationnelle
Exposition réseau	Service public ; allowlist Render 0.0.0.0/0

Écart de configuration : render.yaml déclare notamment MAXIMUS_DEPLOYMENT_MODE, MAXIMUS_CENTRAL_PUBLIC_URL et plusieurs variables DiamanoPay qui ne sont pas toutes visibles dans la configuration effective du service. Le service contient aussi DIAMANOPAY_CALLBACK_URL, qui n'est pas déclarée dans render.yaml. Ce point constitue un risque de dérive opérationnelle et doit être aligné.

L'endpoint de santé expose également le commit applicatif et le protocole de synchronisation. C'est une divulgation d'information faible et probablement volontaire, mais elle doit rester limitée à ces métadonnées.

5. Contrôles applicatifs validés

- Sessions stockées côté serveur sous forme de hash, avec expiration après huit heures.
- Cookies de session Secure, HttpOnly et SameSite=Lax en production.
- Limitation des tentatives de connexion par email et adresse IP.
- Limitation des inscriptions, commandes et retraits financiers.
- Isolation de l'entreprise calculée côté serveur depuis la session.
- Un employé ne peut pas changer de companyId par simple modification de la requête.
- Séparation entre installation centrale et installations dédiées.
- Refus explicite du login central pour les entreprises utilisant une connexion personnalisée.
- Routes d'administration centrale protégées par installation centrale et authentification.
- Headers de sécurité actifs côté API et côté frontend.

6. Plan d'action recommandé

P1	Mettre à jour js-yaml, qs, esbuild et nanoid ; régénérer le lockfile.	Correctif dépendances
P1	Protéger la branche main et exiger les contrôles avant fusion.	Gouvernance GitHub
P2	Activer Dependabot et CodeQL.	Surveillance continue
P2	Aligner les variables Render avec render.yaml et vérifier les secrets DiamanoPay.	Configuration production
P3	Ajouter Permissions-Policy et réduire progressivement la CSP trop large.	Durcissement HTTP
P1	Révoquer et renouveler les identifiants administrateur potentiellement exposés.	Gestion des accès

7. Limites de l'audit

Cet audit est complet pour les contrôles statiques, les dépendances, la configuration GitHub/Render et les vérifications de production non destructives disponibles dans cet environnement. Il ne remplace pas un test d'intrusion externe avec autorisation formelle, un test complet des parcours authentifiés avec comptes de production, ni une revue juridique de conformité. Aucun secret, token, mot de passe ou contenu de base de données n'a été inclus dans le rapport.

Conclusion : MAXIMUS est actuellement opérationnel et dispose de bonnes protections applicatives de base. La réduction du risque passe d'abord par les dépendances, la gouvernance GitHub et l'alignement Render.